TechPost ELEVATED

DC – AI – Routing – Switching – Security – Wireless – Automation

Automate Juniper Apstra with PowerShell

An HPE Juniper Networking Techpost

Shabbir Ahmed

2025-02-24

Contents

1 Automate Juniper Apstra with PowerShell **3**
1.1 Introduction . . . 3
1.1.1 Why PowerShell? . . . 3
1.1.2 Automating Juniper Apstra . . . 4
1.1.3 A Little About PowerShell HTTPS Client/s . . . 4
1.2 Step by Step Explanation of Code for Apstra . . . 4
1.2.1 Authentication with Apstra . . . 4
1.3 Next Step . . . 6
1.3.1 Printing Blueprints as PSObject . . . 6
1.4 Useful links . . . 7
1.5 Glossary . . . 7
1.6 Acknowledgments . . . 7

List of Figures

1 Apstra-PS 3

1 Automate Juniper Apstra with PowerShell

Shabbir Ahmed - 02/24/2025

It's all built into Windows.

Leverage Windows PowerShell to automate Juniper Apstra without installing PowerShell as a language or any libraries (like the HTTP client library).

You can download the script on an enterprise workstation where you access the Apstra Web UI and run it without seeking approvals from the infosec team or installing any third-party modules from the Internet. No extra VMs, languages, or library installation is required from trusted or untrusted sources.



Figure 1: Apstra-PS

1.1 Introduction

This blog is about automating Juniper Apstra with PowerShell. It explains how we can call an Apstra REST API from PowerShell and provides a working sample script at the end of the blog.

1.1.1 Why PowerShell?

Why learn and use another scripting language or learning path while Python is already available with many DevOps features?

I work with multiple large customers, from Service Providers to Enterprises, and I regularly run into issues when requesting a Linux Machine with Python and all the other required libraries. Enabling all those tools required to run a script takes a lot of time, from spinning up a VM to seeking multiple management and security approvals with dependence on multiple teams. Most enterprises don't allow internet access to the infrastructure elements hosted inside a DC. And the installation of libraries becomes troublesome too. Also, customers may refuse the library installation if it's not supported or developed by a trusted source. Projects like this are doomed from the beginning.

On the other hand, PowerShell is built into Microsoft Windows, and most of its basic libraries are pre-installed. The sales pitch here is to run the script almost immediately without waiting for approvals from management and the involvement of different teams.

Just download the script and execute it to accomplish the given task. The same Windows PC used to configure or manage Juniper Apstra can run the PowerShell script to automate Apstra, thus avoiding the need for any special environment.

1.1.2 Automating Juniper Apstra

Apstra uses a REST (REpresentational State Transfer) API, which is an architectural style for delivering APIs that is based on the HTTP specification used by the web. It's important to note that REST is a style, rather than a standard, and it is made up of different standards such as HTTP, XML, URL, and JSON.

REST APIs use URLs (Uniform Resource Locators) to make data available, and they rely on HTTP methods, headers, and other building blocks to facilitate communication between systems. One of the benefits of REST APIs is that they are simple and widely recognized, making them a common starting point for teams when they are first learning about APIs.

When you make an API call, the server receives your request and processes it. If the request is successful, the server will typically return data in response to your request. This data is returned in JSON (JavaScript Object Notation) format.

1.1.3 A Little About PowerShell HTTPS Client/s

PowerShell has a built-in command **Invoke-RestMethod** cmdlet which sends HTTP and HTTPS requests to REpresentational State Transfer (REST) web services that return richly structured data.

PowerShell formats the response based on the data type. For JavaScript Object Notation (JSON) or XML, PowerShell converts or deserializes, the content into [PSCustomObject] objects. These are PS data structures for manipulating data, and they are represented as [key, value] pairs.

In our example, we are also utilizing another cmdlet **Invoke-WebRequest. Invoke-WebRequest** is more of a CLI browser and can be used to manipulate HTML, images, and other web data while **Invoke-RestMethod** is only designed to consume RestAPIs. **Invoke-WebRequest** can do all **Invoke-RestMethod** does and more.

1.2 Step by Step Explanation of Code for Apstra

1.2.1 Authentication with Apstra

Our first interaction with Apstra will be user authentication. Apstra is built around token-based authentication, where a token hash is returned if the provided authentication parameters are correct. This token is used in subsequent API calls.

```
1     $server = 'https://10.85.135.49'
2     $username = 'USERNAME'
3     $password = 'PASSWORD'
4
5     # complete the url for authentionation API
6
7     $Url = $server + "/api/user/login"
8
9     # Prepare HTTP headers
10
11    $headers = @{
12     'Content-Type'='application/json'
13     'Cache-Control'='no-cache'
```

```
14      }
15
16      # authentication REST CALL takes username, password in HTTP Body
17
18      $Body = @{
19         username = $username
20         password = $password
21      }
22
23      Write-Host -NoNewline "Sending Authentication Request "
24
25      # The following HTTP REST call authenticates the user and gets the token.
26
27      try {
28
29         $Result = Invoke-RestMethod -Method 'Post' -Uri $url -Body ($body|ConvertTo-Json) -
               ContentType "application/json" -Headers $headers
30
31      } catch {
32
33         Write-Host "StatusCode:" $_.Exception.Response.StatusCode.value__
34         Write-Host "StatusDescription:" $_.Exception.Response.StatusDescription
35         break
36      }
37
38      # if token is received, we can proceed with further HTTP calls.
39      # if token is not received disconnect and stop further script execution.
40
41      if ($Result.token) {
42
43         Write-Host -ForegroundColor Yellow "Authentication toke received"
44      } else { break }
```

Authentication Request to Apstra from PowerShell

We first build out variables and assign them with values. The variables are self-explanatory, the $server is assigned the IP/Domain of the Apstra Server, $username, $password where we set the username passwords. The part of the API call prepares the complete url.

Next, we prepare the HTTP header and body, the header part passes some important variables to the server while the username and password are passed to the server in the body part of the HTTP request. The body is passed to HTTP server in JSON format.

Finally, we call the Apstra API with HTTP POST method using the following code.

```
1  $Result = Invoke-RestMethod -Method 'Post' -Uri $url -Body ($body|ConvertTo-Json) -ContentType "
       application/json" -Headers $headers
```

The above line is run inside a try-catch code block to better handle the exceptions.

If the username, and password including the IP address are correct, the output will be printed on the screen, this will also print the token received from Apstra, which will be used in further API calls.

> NOTE: the token is only printed for educational purposes and the line must be commented on in real-world scripts.

To run the script please navigate to the start menu and type PowerShell, open PowerShell and navigate to the code then run poweshell.exe script name.

```
1  Sending Authentication Request
2  Authentication token received
3  eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzUxMiJ9.
       eyJ1c2VybmFtZSI6ImFkbWluIiwidXNlcl9zZXNzaW9uIjoiYTU2YjIyNWUtNGQ0MC00MjEzLThlOWItOGUxZjE3ZjhiZWJmIiwiY3Jl
       .5lqgRb3h1rhOqjYdX08hz0We_Ftl8qNXnHq-GuLWVwuBg3ZnNgHb8mE1zO_fhW391QaPqNJAlmB6-wffHzK4Tg
```

1.3 Next Step

Now that we have the token, we can call any other API to fetch/consume the data and process it. The next step is to make an API call and retrieve the data. In the following API call, I will try to retrieve the blueprints and their ids which are configured on the Apstra Server.

First, we prepare the HTTP header for the subsequent API calls. The main change consists in adding the token to the header under a variable named 'AUTHTOKEN'.

Second, we add together everything to make up the URL. The URL contains Apstra IP with Uri /api/blueprints.

The call to

```
$result =(Invoke-WebRequest -Method Get -Uri $url -ContentType "application/json" -Headers
    $headers -usebasicparsing).content | ConvertFrom-Json
```

retrieves the list of running blueprints along with their ids.

Example of API calls to retrieve the blueprints

```
$headers = @{

    'AUTHTOKEN' = $Result.token
    'Content-Type'='application/json'
    'Cache-Control'='no-cache'
    'accept' = 'application/json'
    }

   $url = $server + "/api/blueprints"

   try {

      $result =(Invoke-WebRequest -Method Get -Uri $url -ContentType "application/json" -Headers
           $headers -usebasicparsing).content | ConvertFrom-Json

   } catch {
      Write-Host $_.Exception.Response
   }
```

1.3.1 Printing Blueprints as PSObject

The above API call returns a JSON object which is converted to [PSCustomObject] by using **ConvertFrom-Json**. [PSCustomObject] is a PowerShell representation of JSON/XML and it makes further script processing easier. Similarly, Python converts JSON to dictionaries, in PS we convert the JSON output to a [PSCustomObject].

```
# Iterate through all the blueprint returned.

foreach ($item in $result.items) {

    foreach($bp in $item) {

      Write-Host "BluePrint ID:" $bp.id " for " $bp.label
    }
}
```

Example of API calls to output

```
BluePrint ID: 6e780fe4-e10e-4758-b1c9-ad409dd9f2e4 for IPCLOS
BluePrint ID: 1a9f6a1a-6afa-4d37-8076-0a4e6edb5911 for 5-STAGE-CLOS
```

The above text is the output of the script we ran, the output mentions the blueprints configured on Apstra.

Similarly, we can make API calls to retrieve data, process it, and use it as input for further API calls.

Or we can retrieve data and generate custom reports from Apstra.

1.4 Useful links

- Apstra Automation and basics of Rest API: https://cloudlabs.apstra.com/labguide/Cloudlabs/4.2.1/labapi/labapibasics.html
- PowerShell web client: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/invoke-restmethod?view=powershell-7.4

1.5 Glossary

- API: Application programming interface
- HTTP(S): Hypertext Transfer Protocol Secure
- JSON: JavaScript Object Notation
- PS: PowerShell
- REST: Representational State Transfer
- URI: Uniform Resource Identifier
- URL: Uniform Resource Locator (the address of a web page)
- VM: Virtual Machine
- XML: Extensible Markup Language

1.6 Acknowledgments

Thanks to Jessica Garrison and Bill Wester



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.
Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.
Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

HEWLETT PACKARD ENTERPRISE

HPE.com



